

Temperature Data Acquisition Using Arduino and Noise Reduction via Adaptive Filtering

Abhik Haldar

Department of Electrical Engineering
Indian Institute of Technology Ropar
Rupnagar, India
2025eem1001@iitrpr.ac.in

Shinguingam Phaomei

Department of Electrical Engineering
Indian Institute of Technology Ropar
Rupnagar, India
2025eem1010@iitrpr.ac.in

Simrandeep Kaur

Department of Electrical Engineering
Indian Institute of Technology Ropar
Rupnagar, India
simrandeep.22eez0007@iitrpr.ac.in

Shyam Jivan Patil

Department of Electrical Engineering
Indian Institute of Technology Ropar
Rupnagar, India
shyam.24eez0008@iitrpr.ac.in

Satyam Agarwal

Department of Electrical Engineering
Indian Institute of Technology Ropar
Rupnagar, India
satyam@iitrpr.ac.in

Abstract—Temperature measurement plays an important role in the field of industrial automation, environmental monitoring, and embedded sensing systems. Nevertheless, measurements made by sensors suffer from the influence of electrical interference and external noise, which decreases the reliability of the obtained data. Traditional fixed parameter filters cannot cope with such dynamic noise variations, thus there appears a need in the development of adaptive filtering techniques, allowing for adjustment to constantly changing signal parameters. In response to the posed problem, this paper develops a temperature data acquisition system with noise suppression using an Arduino Uno board and DS18B20 temperature sensor with the application of adaptive noise removal techniques utilizing Least Mean Squares (LMS) and Recursive Least Squares (RLS) algorithms. Artificial Gaussian noise is added to the obtained temperature data in order to mimic practical application conditions. Adaptive algorithms are compared according to the Mean Square Error (MSE), Signal-to-Noise Ratio (SNR) and convergence properties. As a result of experiments, it is shown that LMS and RLS algorithms effectively remove the noise from measurements, where LMS algorithm demonstrates a low computational complexity, whereas RLS shows better convergence and tracking properties.

Keywords— Arduino, Temperature Sensor, Adaptive Filtering, LMS Algorithm, Noise Reduction, Signal Processing.

I. INTRODUCTION

Temperature measurement is an important aspect in many different areas including automation, environment monitoring, health care system, and embedded sensor systems. Accurate data on temperature is required for various processes of monitoring, system operation, equipment protection, etc. However, temperature measurements that are acquired using different sensor systems can be influenced by noise that appears due to various disturbances, interference, non-linear behavior of the sensor, etc., thus reducing the measurement quality and accuracy [1]. Hence, improvement of the sensors measurements became an interesting research topic in embedded signal processing.

Moving-average and fixed-coefficient digital filters are well-known conventional filters that are used for measurement

noise suppression. The reason of their popularity lies in their simplicity but they use fixed parameters and cannot be adapted to changing noise environment. Adaptation is provided by the adaptive filtering technique that uses the input signal statistics to update filter coefficients, thus providing better suppression of noise without signal distortion [2]. Many different algorithms of adaptive filtering exist today but LMS and RLS algorithms are the most popular due to their simplicity and robustness.

In view of the above problems, this paper introduces a real-time temperature data acquisition system using an Arduino Uno and a DS18B20 temperature sensor with adaptive noise suppression through the LMS and RLS algorithms. Artificial noise with Gaussian distribution is added to the acquired temperature data so as to simulate real-world disturbances in measurements and set up a controllable scenario for performance evaluation of the adaptive filters. The performance of both algorithms is measured in terms of MSE, SNR, estimation error convergence and adaptive weights convergence. The experimental results have revealed the fact that adaptive filtering helps to improve the quality of acquired temperature data. In addition, the comparison shows the advantages and disadvantages of both algorithms which include the simplicity of computations and fast convergence and better estimation in the case of the RLS algorithm[9].

Apart from the LMS filter, the RLS (Recursive Least Squares) algorithm was used in this research to compare its performance to the LMS filter. The RLS converges much faster than the LMS filter and gives a much better output at a cost of high computation.

II. SYSTEM ARCHITECTURE

A. Hardware Components

The experimental setup consists of:

- Arduino Uno
- DS18B20 Temperature Sensor

- OneWire Interface
- Serial Communication Link

B. System Block Diagram

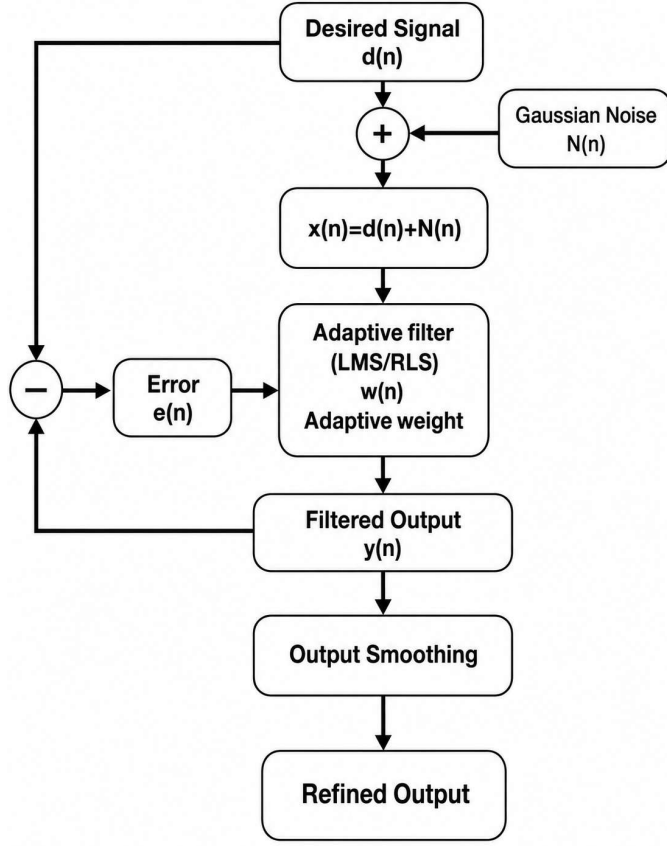


Fig 1: Block diagram of the temperature noise cancellation system.

In Fig 1 the block diagram presents the adaptive noise reduction framework which is applied to the temperature signal in real time. The temperature signal, $d(n)$, is mixed with a Gaussian noise, $N(n)$, to form the noisy input signal, $x(n)$. The adaptive filter implemented by the use of the LMS and RLS algorithms is applied to the noisy input signal, $x(n)$, to obtain the filtered output signal, $y(n)$. The difference between the original temperature signal, $d(n)$, and the filtered output signal, $y(n)$, is calculated as the estimated error signal, $e(n) = d(n) - y(n)$ and this error signal is used for the iterative adjustment of the weights of the adaptive filter. In order to increase the stability of the signal output and reduce the variation of its output, a smoothing process is also applied to the output signal, which results in the smoothed output of the temperature signal. The final smoothed and improved temperature signal, $T_f(n)$, represents the denoised temperature signal with higher accuracy and lower noise.

III. METHODOLOGY

A. Temperature Data Acquisition

The temperature data acquisition system consists of a DS18B20 digital temperature sensor interfaced with an Arduino Uno microcontroller. The sensor continuously measures ambient temperature and transmits the readings to the Arduino through the OneWire communication protocol.

The acquired temperature signal is considered the desired signal, $d(n)$ (actual sensor measurement)[8][10].

This desired signal serves as the reference input for the adaptive filtering process.

B. Noise Injection

To simulate practical measurement disturbances, a Gaussian noise with mean 0 and variance σ^2 is introduced into the desired temperature signal.

The noisy input signal is represented as:

$$x(n) = d(n) + N(n) \quad (1)$$

where $x(n)$ is the noisy input signal, $d(n)$ is the desired temperature signal acquired from the sensor, and $N(n)$ represents the artificially generated Gaussian noise added to simulate measurement disturbances.

This noisy signal is applied as the input to the adaptive filter.

C. LMS Adaptive Filtering

The Least Mean Squares (LMS) algorithm is employed to estimate and remove noise from the corrupted signal.

The filter output is given by:

$$y(n) = w_0x(n) + w_1x(n-1) + w_2x(n-2) \quad (2)$$

where $y(n)$ is the filter output, $w(n)$ represents the adaptive filter weight vector, and $x(n)$ is the noisy input signal[6].

D. RLS Adaptive Filtering

In order to obtain faster convergence and greater estimation accuracy, the algorithm known as Recursive Least Squares (RLS) was implemented for use. The RLS algorithm uses the inverse of the correlation matrix of the input signal to perform the updating of the filter weights which provides much better suppression of noise compared to the LMS approach.

E. Error Estimation

The estimation error is calculated as the difference between the desired signal and filter output:

$$e(n) = d(n) - y(n) \quad (3)$$

where $e(n)$ represents the estimation error.

The objective of the LMS algorithm is to minimize this error iteratively.

F. Adaptive Weight Update

The filter coefficient is updated using the LMS adaptation rule:

$$w(n+1) = w(n) + \mu e(n)x(n) \quad (4)$$

where μ is the learning rate, and $w(n+1)$ represents the updated adaptive filter weight vector. This allows the filter to adapt dynamically to changing noise conditions[9].

The adaptive updated weight for RLS algorithm is given by:

$$w(n+1) = w(n) + K(n)e(n) \quad (5)$$

where $w(n+1)$ denotes the updated adaptive filter weight vector after the $(n+1)^{\text{th}}$ iteration, $w(n)$ is the adaptive filter weight vector at the n^{th} iteration, $K(n)$ represents the gain vector that controls the magnitude of the weight update, and $e(n)$ is the estimation error.

G. Output Smoothing

To further improve stability and reduce fluctuations in the filtered temperature signal, an additional smoothing operation is applied:

$$T_f(n) = 0.8T_f(n-1) + 0.2T_c(n) \quad (6)$$

where $T_f(n)$ denotes the refined filtered temperature, $T_c(n)$ represents the current filtered temperature estimate, and α is the smoothing factor satisfying $0 < \alpha < 1$. This smoothing stage improves the consistency and stability of the filtered temperature signal by reducing residual fluctuations.

This stage improves output consistency.

H. Overall Algorithm

The proposed methodological framework starts with the acquisition of real-time temperature readings using the DS18B20 digital temperature sensor interfaced with the Arduino Uno, where the acquired signal is considered the desired temperature signal, $d(n)$. Gaussian noise is artificially added to the desired signal to generate the noisy input signal, $x(n)$. This distorted signal is then processed using the LMS and RLS adaptive filtering algorithms to obtain the filtered output $y(n)$. The estimation error $e(n)$, is computed as the difference between the desired signal and the filtered output. The adaptive filter coefficients are iteratively updated according to the weight update equations of the LMS and RLS algorithms. Finally, the filtered output is smoothed to eliminate any remaining fluctuations in the estimated temperature signal. The performance of the proposed method is evaluated by comparing the LMS and RLS algorithms in terms of mean square error (MSE), signal-to-noise ratio (SNR), convergence behavior, and adaptive weight evolution.

IV. IMPLEMENTATION

A. Hardware Implementation

The proposed method is applied by means of Arduino Uno microcontroller connected to DS18B20 digital temperature sensor as shown in Fig.2. It is attached to OneWire network which records temperature in real time. The data obtained in

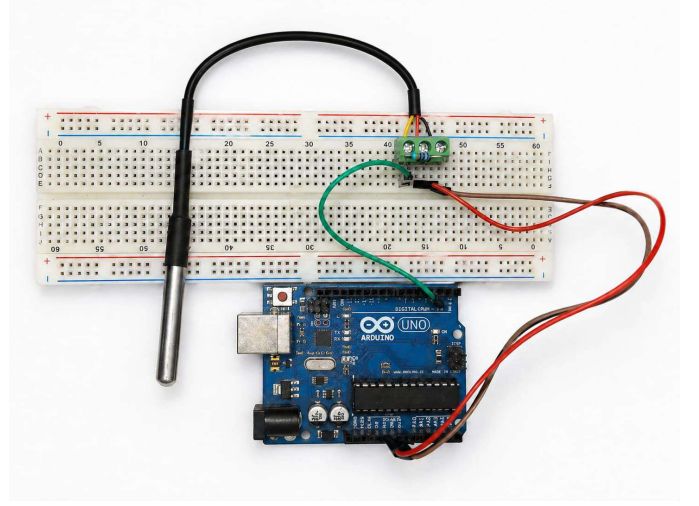


Fig 2: Hardware setup of the Arduino-Based Temperature Measurement System

this way are transmitted in a serial way for further analysis and control.

B. Software Implementation

In order to implement the software of the proposed system, the Arduino IDE development platform has been used to perform real-time temperature acquisition, adaptive noise cancellation, and visualization of results. First of all, temperature values are acquired by using the DS18B20 digital temperature sensor and the Arduino Uno board. Artificial Gaussian noise is injected in the measurement data in order to mimic the real situation and test the effectiveness of the developed adaptive filters. Then, the contaminated signal is filtered by applying the LMS and RLS adaptive filters, in which filter coefficients are updated based on the corresponding adaptation formula. At the output of the adaptive filters, the signal smoothing algorithm is performed to reduce possible fluctuations. Finally, the input, corrupted, and filtered temperature signals and estimation error, as well as adaptive filter coefficients, are visualized on the serial monitor.

C. LMS Parameter Selection

The LMS adaptive filter was developed by applying an initial weight of 0.9, learning rate (μ) of 0.005, Gaussian noise range of $\pm 4^\circ\text{C}$, and sampling time of 1 s. The learning rate was set at this level in order to ensure fast but stable convergence, while the noise range represents a realistic level of measurement interference observed in practice. In addition, the sampling time ensures continuous measurement of the temperature, which allows the adaptive filter to effectively adapt to changes in the input signal. Thus, all of these parameters were set to ensure effective filtering and convergence.

D. Real-Time Data Processing

At every sampling point, obtained temperature measurements are affected by artificial noise, and the obtained measurements are processed using an LMS adaptive filter. The

adaptive weight is updated as long as the error signal exists to reduce estimation error. The smoothing process is carried out on the filtered output to enhance the stability of the output.

E. Serial Output Monitoring

The processed data is displayed through the serial monitor in the following format:

Original → Noisy → Refined → Error → Weight

This output enables comparison of raw sensor data, noisy measurements and filtered results.

V. RESULTS AND ANALYSIS

A. Real-Time Temperature Signal Comparison

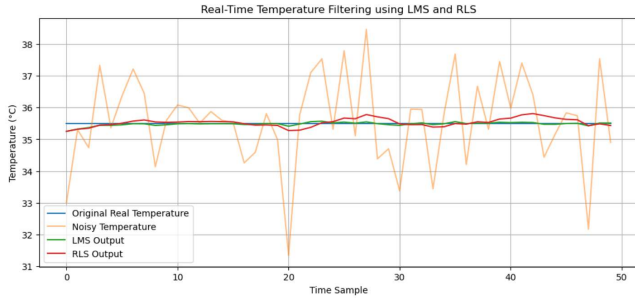


Fig 3: Desired, Noisy and Filtered Temperature Graph

Fig.3 presents a comparison of the original temperature signal taken from the DS18B20 temperature sensor to that of an artificially-noisy version of that signal and to that of the temperature signal after application of the LMS and RLS adaptive filter algorithms.

The artificial noisy signal varies considerably due to the addition of Gaussian type noise. The LMS and RLS filters are successful deburring noise from the original temperature signal. The RLS filter has slightly better tracking performance than the LMS filter.

Mean Square error Performance Analysis The Mean Square

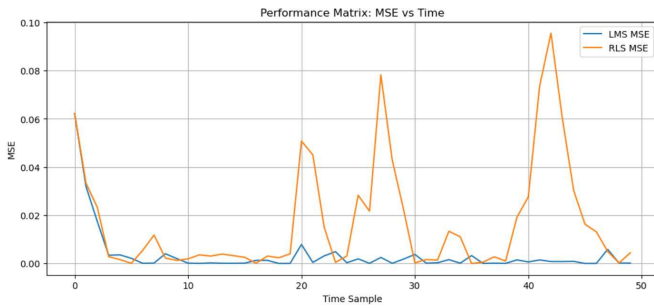


Fig 4: Temperature Datasheet

Error (MSE) performance of LMS and RLS algorithms is illustrated in Fig. 4.

The MSE is computed as

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N e^2(n) \quad (7)$$

The results show that the RLS algorithm converges faster and achieves lower steady-state error compared to the LMS algorithm. This confirms the superior convergence characteristics of the RLS adaptive filter.

B. Error Convergence Behavior



Fig 5: Error convergence LMS vs RLS

Fig. 5 illustrates the estimation error convergence for LMS and RLS filtering.

Both algorithms have greater estimation error values because they are adjusting from the beginning weight values at first as well. The size of the Error decreases significantly as the iterations increase. The RLS stabilizes faster than the LMS and therefore has a faster stabilizing time[7].

C. Adaptive Weight Convergence Analysis

The convergence of adaptive filter coefficients for LMS and RLS algorithms is shown in Fig. 6 and Fig. 7 respectively.

Adaptive weight values reach their near-steady levels after some time of repeating process until the two filtering algorithms converge. The RLS has rapid adaptive weight stabilization compared to the LMS because of its recursive optimization nature.[3]

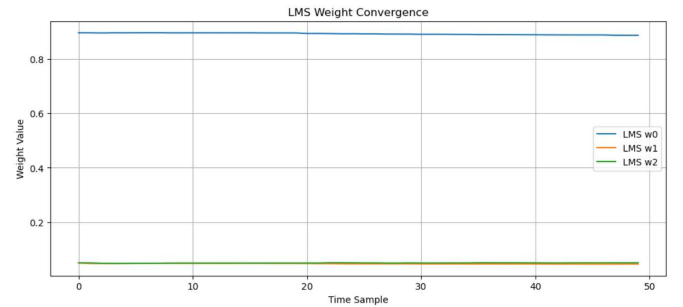


Fig 6: LMS weight convergence

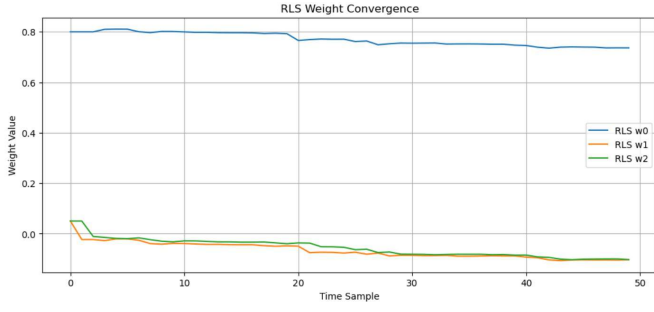


Fig 7: RLS weight convergence

D. Signal-to-Noise Ratio

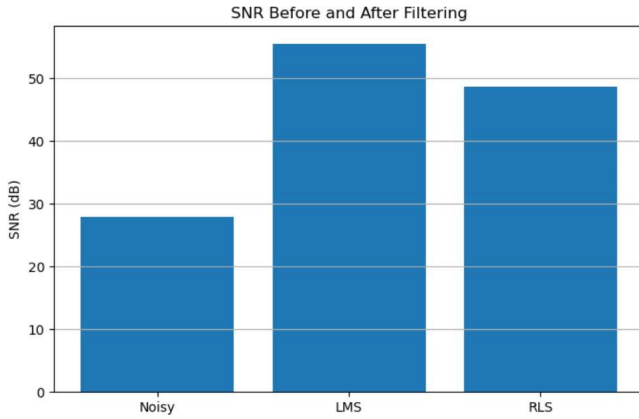


Fig 8: SNR before and after filtering

Signal-to-noise ratio (SNR) improvement is evaluated to quantify the effectiveness of the adaptive filtering process. The SNR is calculated using

$$\text{SNR} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right) \quad (8)$$

The experimental results obtained are as follows:

- Before filtering: SNR = 27.88 dB
- After LMS filtering: SNR = 55.58 dB
- After RLS filtering: SNR = 48.67 dB

The results demonstrate both of the adaptive filters provide improved signal quality compared to the noisy input signal and the RLS filter produced a significant improvement in signal quality.

E. Comparative Performance Summary

Table I summarizes the performance comparison between LMS and RLS filtering algorithms.

On the whole, the above experimental findings show that both LMS and RLS adaptive filters provide a significant improvement in the performance of the acquired temperature signal by reducing the measurement noises. Although the LMS filter provides a computationally simple solution applicable to real-time embedded systems, the RLS algorithm offers fast

TABLE I: Performance Comparison Between LMS and RLS Algorithms

Parameter	LMS	RLS
Convergence Speed	Moderate	Fast
Steady-State Error	Low	Very Low
Computational Complexity	Low	High
SNR Improvement	Higher in this experiment	Slightly lower
Stability	Good	Excellent

convergence and more accurate estimation despite its complex computations. In this respect, the choice of adaptive filter algorithm should depend on the particular application needs.

VI. CONCLUSION

In this study, a real-time temperature monitoring and noise suppression system was designed with the Arduino Uno board and DS18B20 temperature sensor. The developed temperature data was sent to Python for further processing where Gaussian noise was added to simulate real-world measurement noise. To reduce the noise effect, both adaptive filtering algorithms (LMS and RLS) were tested and compared. Results from Experimental shows that combination of two filters significantly improved the quality of temperature signal. There was a lot of variation in the raw noisy temperature signal when the measured temperature was compared with the filtered signals. but by applying the filters to create the filtered signals, the filters produced an output that more.

MSE, SNR, error variation and convergence weight were all used as metrics to confirm that adaptive filtering is beneficial in real time signal processing to sensor signals. According to the reports, the RSL filter demonstrated faster convergence and better tracking ability during adaptation. whereas the LMS filter requires less computing horsepower and is simpler to use. Thus, LMS is ideal where simplicity of installation is required while RLS would be preferred where fast convergence and high accuracy in estimate are required.

That's all. In addition, the smoothing stage reduced small and minor fluctuations that remained present within the filtered signal. Therefore, it is applied to the final temperature estimate to make it more stable for real time monitoring. The developed system demonstrated how adaptive filtering can increase the reliability of temperature measurement in sensor-based embedded systems, and also proved that by using a low cost Arduino-based measurement system, along with appropriate signal processing methods can improve the accuracy obtained from temperature sensor measurements.

VII. FUTURE WORK

While this system shows good noise reduction using either LSM or RLS adaptive filtering algorithms, there is future work that could make the system even better. For instance, advanced adaptive filtering methods like Normalized LSM (NLMS) or adaptive filters with variable step sizes could further enhance the performance of both filters in terms of their ability to converge under variable noise conditions. In this research we added artificial Gaussian noise as a way to test how our filter performed. It is possible to use actual environmental noise

and hardware-related noise in future research to provide more accurate performance analysis.

IoT-based wireless communication modules can be included with this system for remote temperature monitoring and cloud-based data logging to improve the system further[11]. This would allow the continuous, real-time observation of temperature measurements, as well as storage of those temperature measurements, for large-scale temperature monitoring applications. In addition, there is an opportunity to investigate the potential of implementing adaptive filtering algorithm hardware-level design of FPGA platforms for use in the high speed, real-time signal processing environment of embedded monitoring systems.

REFERENCES

- [1] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2014.
- [2] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1985.
- [3] Arduino, "Arduino Uno Rev3 Datasheet," Arduino Documentation. [Online]. Available: <https://www.arduino.cc>
- [4] Maxim Integrated, "DS18B20 Programmable Resolution 1-Wire Digital Thermometer Datasheet," Maxim Integrated Products.
- [5] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed. New York, NY, USA: McGraw-Hill, 2011.
- [6] A. H. Sayed, *Adaptive Filters*. Hoboken, NJ, USA: John Wiley Sons, 2008.
- [7] A. Spanias, "A Brief Survey of Time- and Frequency-Domain Adaptive Filters," *Proc. IEEE International Symposium on Intelligent Signal Processing*, 2016.
- [8] R. Saha et al., "A Working Prototype Using DS18B20 Temperature Sensor for Real-Time Health Monitoring Applications," *IEEE-related sensor application reference* (2021).
- [9] B. Widrow, J. M. McCool, M. G. Larimore, and C. R. Johnson, Jr., "Stationary and Nonstationary Learning Characteristics of the LMS Adaptive Filter," *Proceedings of the IEEE*, vol. 64, no. 8, pp. 1151–1162, Aug. 1976.
- [10] M. N. Islam, M. A. Rahman, et al., "Arduino-Based Smart Temperature Monitoring System Using DS18B20 Sensor," *IEEE International Conference on Electrical, Computer and Communication Engineering*.
- [11] A. Kumar et al., "Internet of Things Based Smart Temperature Monitoring System," *IEEE International Conference on Smart Computing and Electronic Enterprise (ICSCEE)*.